

# **Fabryka Abstrakcyjna - Biblioteka Projekt**

## Spis treści

|      |                                           |   |
|------|-------------------------------------------|---|
| 1.   | Fabryka abstrakcyjna .....                | 3 |
| 1.1. | Publication .....                         | 3 |
| 1.2. | PublicationDB .....                       | 3 |
| 1.3. | GUIFactory .....                          | 4 |
| 2.   | Wyjątek InvalidPublicationException ..... | 4 |
| 3.   | Enum – Topic .....                        | 5 |
| 4.   | Testowanie.....                           | 5 |
| 5.   | Users.....                                | 6 |
| 6.   | Menu w mainie.....                        | 7 |
| 7.   | Baza danych.....                          | 8 |

# 1. Fabryka abstrakcyjna

Projekt jest wykonany schematem fabryki abstrakcyjnej. Aplikacja zawiera 3 podstawowe interfejsy: Publication, PublicationDB, GUIFactory.

## 1.1. Publication

Jest to interfejs publikacji (produktu w bibliotece). Zawiera jedną metodę abstrakcyjną displayInfo(), która służy do wyświetlania informacji o produkcie.

```
public interface Publication {  Jnicob
    void displayInfo(); 7 usages 3 implement
}
```

## 1.2. PublicationDB

Jest to interfejs do obsługi operacji na bazie danych związanych z produktami z biblioteki. Zawiera 6 metod abstrakcyjnych:

- createTable() – metoda tworząca tabelę do przechowywania danej publikacji,
- addpublication() – metoda dodająca publikację do bazy danych
- getAllPublications() – metoda zwracająca wszystkie publikacje z danej tabeli,
- rentPublication() – metoda wypożyczająca daną publikację przez danego użytkownika (w założeniu, że dany użytkownik oraz publikacja istnieją oraz, że użytkownik jeszcze jej nie wypożyczył),
- returnPublication() – metoda zwracająca daną publikację przez danego użytkownika (zakładając, że użytkownik już wcześniej ją wypożyczył),
- deletePublication() – metoda usuwająca publikację z bazy danych po ID

```
public interface PublicationDB<T extends Publication> { // T oznacza jakik
    void createTable(); no usages 3 implementations  Jnicob
    void addPublication(T publication); 4 usages 3 implementations  Jnicob
    ArrayList<T> getAllPublications(); 5 usages 3 implementations  Jnicob
    void rentPublication(int publicationID, int userID); 3 usages 3 implemen
    void returnPublication(int publicationID, int userID); 3 usages 3 implem
    void deletePublication(int publicationID); 4 usages 3 implementations  Jn
}
```

### 1.3. GUIFactory

Interfejs służący do obsługi fabryk abstrakcyjnych.

```
public interface GUIFactory { 4 usages 3 implementations 2 Jnicob +1
    Publication createPublication() throws InvalidPublicationException; 1 usage
    PublicationDB createPublicationDB(); 18 usages 3 implementations 2 Jnicob
}
```

## 2. Wyjątek InvalidPublicationException

Jest to wyjątek walidujący poprawność danych do publikacji. Zawiera dwie metody:

- validateString() – sprawdza, czy wartość typu String nie jest pusta,
- validatePositive() – sprawdza czy wartości typu int są dodatnie (zawsze muszą być dodatnie, bo są to wartości typu: ilość w magazynie, ilość dostępnych języków nagrania, itd.)

```
public class InvalidPublicationException extends Exception { 2 Szazam07
    public InvalidPublicationException(String message) { super(message); }
    public InvalidPublicationException(String message, Throwable cause) { super(message, cause); }

    // Static helper to validate publication data
    @Contract("_, null -> fail")
    public static void validateString(String fieldName, String value) throws InvalidPublicationException {
        if (value == null || value.trim().isEmpty()) {
            throw new InvalidPublicationException(fieldName + " cannot be empty or null.");
        }
    }

    public static void validatePositive(String fieldName, int value) throws InvalidPublicationException {
        if (value < 0) {
            throw new InvalidPublicationException(fieldName + " cannot be negative.");
        }
    }
}
```

Jako że jest to wyjątek do walidacji podanych danych, to rzucają go np. konstruktory.

```
public BookObj() throws InvalidPublicationException { 1 usage 2 Jnicob
    this( id: 0, title: "", genre: "", Date.valueOf(LocalDate.now()), authorID: 0, publisherID: 0, quantityInStock: 0, language: "", pagesAmount: 0);
}
```

```
try{
    p = new AudiobookObj(audiobookTitle, audiobookGenre, au
    pDB.addPublication(p);
    System.out.println("Audiobook added successfully!");
}catch(InvalidPublicationException e){
    System.out.println(e.getMessage());
    throw new RuntimeException(e);
}
```

### 3. Enum – Topic

Projekt zawiera wbudowany enum Topic w klasie MagazineObj. Służy jako pole z konkretnymi możliwymi wartościami do pola topic w klasie MagazineObj.

```
public enum Topic { 11 usages  Szazam07
    TECHNOLOGY, 3 usages
    HEALTH, no usages
    TRAVEL, no usages
    BUSINESS, no usages
    LIFESTYLE no usages
}

private int id; 3 usages
private String title; 3 usages
private Topic topic; 3 usages
private Date releaseDate; 3 usages
private int publisherID; 3 usages
private int quantityInStock; 3 usages
private int articlesAmmount; 3 usages
```

### 4. Testowanie

Projekt zawiera 3 pliki do testowania: BookObjTest, MagazineObjTest, AudiobookObjTest. Pojawiają się w nich metody służące do testowania tworzenia obiektów (np. testMagazineCreation()),

```
@Test  Szazam07 +1
void testMagazineCreation() throws InvalidPublicationException {
    MagazineObj mag = new MagazineObj( title: "Business Weekly", MagazineObj.Topic.LI
    assertEquals( expected: "Business Weekly", mag.getTitle());
    assertEquals( expected: "BUSINESS", mag.getTopic());
    assertEquals( expected: 120, mag.getQuantityInStock());
}
```

testowania walidacji (np. `testInvalidMagazineThrowsException`),

```
@Test  Szazam07 +1
void testInvalidMagazineThrowsException() {
    assertThrows(InvalidPublicationException.class, () -> {
        new MagazineObj( title: "", MagazineObj.Topic.HEALTH, Date.valueOf(LocalDate.now(
    });
    assertThrows(InvalidPublicationException.class, () -> {
        new MagazineObj( title: "Tech Guide", MagazineObj.Topic.TECHNOLOGY, Date.valueOf(
    });
}
```

Lub dodawania zmiany wartości ilości w obiektach (np. `testUpdateMagazineQuantity()`)

```
@Test  Szazam07
void testUpdateMagazineQuantity() throws InvalidPublicationException
    MagazineObj mag = magazines.get(0);
    int oldQty = mag.getQuantityInStock();
    int newQty = oldQty + 20;
    // symulacja aktualizacji danych
    magazines.set(0, new MagazineObj(
        mag.getId(),
        mag.getTitle(),
        mag.getTopic(),
        mag.getReleaseDate(),
        mag.getPublisherID(),
        newQty,
        mag.getArticlesAmmount()
    ));
    assertEquals(newQty, magazines.get(0).getQuantityInStock());
}
```

## 5. Users

Projekt zawiera motyw posiadania użytkowników z username, emailiem i hasłem. Hasło jest wykorzystywane podczas wypożyczania i zwracania książek jako użytkownik dla walidacji. Po podaniu prawidłowego hasła dopiero możliwe jest zwracanie lub wypożyczanie. Użytkownicy nie są zawarci w ramach fabryki abstrakcyjnej, gdyż nie są na nich wykonywane żadne operacje w programie, ani nie dodajemy lub usuwamy ich z bazy w trakcie działania programu. Wszystko związane z użytkownikami znajduje się w obiektach: `User`, `UserDB`.

## 6. Menu w mainie

W main.java zostało stworzone menu za pomocą switch case. Użytkownik wybiera poprawną wartość i na jej podstawie obsługuje odpowiednie operacje w bazie danych (dodawanie, usuwanie, wyświetlanie danych, wypożyczanie/zwracanie publikacji). Program sterowany za pomocą zmiennych factory (zmienna od fabryk), p (zmienna od publikacji) oraz pDB (zmienna od obsługi baz danych), zmienna typu Scanner służąca do przyjmowania danych od użytkownika w czasie działania programu, 3 arraye do przechowywania zbiorów danych publikacji z bazy, zmienne do przechowywania ID, podanych przez użytkownika oraz zmienna running typu boolean do sprawdzania czy program ma się nadal działać, czy się skończyć.

```
Scanner keyboard = new Scanner(System.in);
GUIFactory factory;
Publication p;
PublicationDB pDB;

// tutaj menu jakies ze switchem
Scanner scanner = new Scanner(System.in);
ArrayList<MagazineObj> magazinesTempArray;
ArrayList<BookObj> booksTempArray;
ArrayList<AudiobookObj> audiobooksTempArray;
int publicationID, userID;
boolean running = true;
```

```
while (running) {
    System.out.println("\n== Publication Management Menu ==");
    System.out.println("1. Add Magazine");
    System.out.println("2. View All Magazines");
    System.out.println("3. Sort Magazines by Title");
    System.out.println("4. Rent a Magazine");
    System.out.println("5. Return a Magazine to Stock");
    System.out.println("6. Delete a Magazine");
    System.out.println("7. Add Book");
    System.out.println("8. View All Books");
    System.out.println("9. Sort Books");
    System.out.println("10. Rent a Book");
    System.out.println("11. Return a Book to Stock");
    System.out.println("12. Delete a Book");
    System.out.println("13. Add Audiobook");
    System.out.println("14. View All Audiobooks");
    System.out.println("15. Rent an Audiobook");
    System.out.println("16. Return an Audiobook to Stock");
    System.out.println("17. Delete an Audiobook");
    System.out.println("18. Exit");
    System.out.print("Choose an option: ");
```

## 7. Baza danych.

Program obsługuje bazę danych bookshopdb. Zawiera obiekt do obsługi połączenia DBConnection. Ten obiekt zawiera metody do tworzenia i zwracania połączenia z bazą danych.

```
public class DBConnection { // klasa ogólna, która służy do inicjacji połączenia z bazą danych

    // dane do połączenia
    private static final String HOST_URL = "jdbc:mysql://localhost:3306/"; 2 usages
    private static final String DB_NAME = "bookshopdb"; 2 usages
    private static final String USER = "root"; 2 usages
    private static final String PASSWORD = ""; 2 usages

    public static void initDatabase() { // metoda inicjująca połączenie 1 usage 2 Jnicob
        try (Connection conn = DriverManager.getConnection(HOST_URL, USER, PASSWORD);
            Statement stmt = conn.createStatement()) {
            stmt.executeUpdate( sql: "CREATE DATABASE IF NOT EXISTS " + DB_NAME);
            System.out.println("Baza danych jest gotowa");
        } catch (SQLException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

```
public static Connection getConnection() throws SQLException { // metoda zwracająca połączenie
    String urlWithDB = HOST_URL+DB_NAME+"?serverTimezone=UTC";
    return DriverManager.getConnection(urlWithDB, USER, PASSWORD);
}
```